

Allow initializing aggregates from a parenthesized list of values

Abstract

This paper proposes allowing initializing aggregates from a parenthesized list of values; that is, `Aggr(val1, val2)` would mean the same thing as `Aggr{val1, val2}`. This is a language fix for the problem illustrated in [N4462](#).

Revision history

- P0960R0 - initial version
- P0960R1 - this version, wording created based on EWG feedback from Jacksonville 2018

Jacksonville 2018 discussion feedback

- Do we want more work in the direction depicted in p0960r0? 12 | 11 | 7 | 1 | 1
- Restrict the change to just dependent types? 1 | 1 | 2 | 13 | 14
- Should designated initializers be supported? 1 | 3 | 12 | 11 | 3
- Should raw arrays be supported? 4 | 14 | 11 | 2 | 1
- Incorporate ban brace-initializing an object if it would be plausible that it would use a deleted constructor? 11 | 14 | 5 | 0 | 3

This revision implements supporting parenthesized initialization for aggregates including arrays, without support for designated initializers. The matter of initialization and deleted constructors is handled by separate paper(s).

Design principles

- Any existing meaning of `A(b)` should not change.
- Parenthesized initialization and braced-initialization should be as similar as possible.
 - One difference remains: for an aggregate `A`, `A{b}` will not convert `b` to `A`, unless `b` is of type `A` or type derived from `A`. `A(b)` will convert even in other cases. This difference applies only to aggregates.

Wording

In [dcl.init]/17, edit as follows:

~~Otherwise, if the destination type is an array, the program is ill-formed.~~

In [dcl.init]/17.6.2, edit as follows:

Otherwise, if the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same class as, or a derived class of, the class of the destination, constructors are considered. The applicable constructors are enumerated (16.3.1.3), and the best one is chosen through overload resolution (16.3). The constructor so selected is called to initialize the object, with the initializer expression or expression-list as its argument(s). If no constructor applies and the destination type is not an aggregate, or the overload resolution is ambiguous, the initialization is ill-formed.

After [dcl.init]/17.6.3 and before [dcl.init]/17.7, insert a new top-level bullet as follows:

If the destination type is an aggregate and the initialization is direct-initialization and no viable constructor was found above (17.6.2), the object is list-initialized as if the initializer was a (non-parenthesized) braced-init-list containing the expression-list. [Note: designators are not permitted. - end note]

Drafting note: the note in the above wording can easily be left out - the grammar of a parenthesized initialization does not allow designators. The note is added just for explanatory clarity.